

1. Project Context & Static Setup

Project Goal: My app tracks tasks across kanban categories (To Do, In Progress, Done).

Static Files: - CSS, JS, and images are in `tasks/static/tasks/`.
- Linked in `base.html` with `{% static 'tasks/css/style.css' %}`. - Theming uses Illinois Blue and Orange.

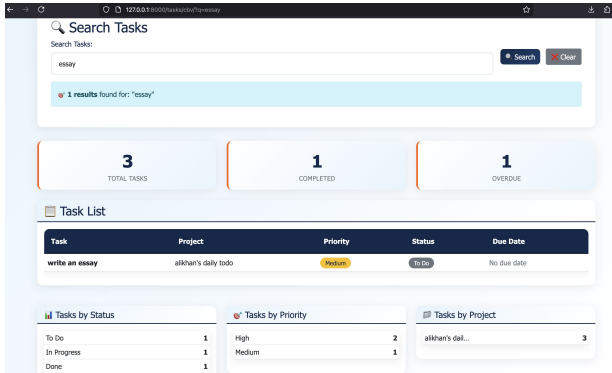


Figure 1: UI with Illinois Theme

2. Chart & Visualization

What it shows: Task distribution by status, priority, and project.

How it's made (Matplotlib):

```
# tasks/views.py
```

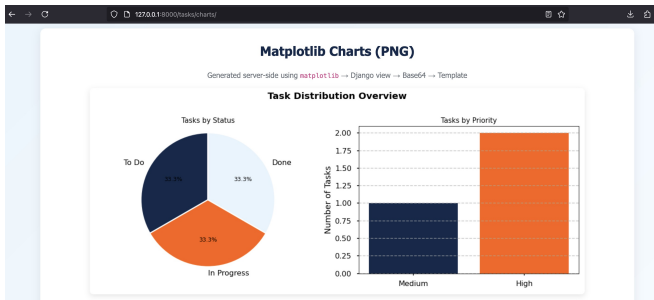
```
fig, (ax1, ax2) = plt.subplots(1, 2)
```

```
ax1.pie(counts, labels=labels)
```

```
buffer = io.BytesIO()
```

```
plt.savefig(buffer, format='png')
```

```
b64 = base64.b64encode(buffer.getvalue())
```



3. Aggregation Summary

ORM Query Logic: I used `annotate()` to count tasks grouped by different fields.

Example: Tasks per project

```
Project.objects.annotate(task_count=Count('tasks'))
```

Stats Displayed: - Total tasks and completed tasks are shown as badges. - Charts visualize tasks per project, status, and priority. - This helps users quickly see the project's overall progress.

4. Reflection

What I Learned: “I understood how `{% static %}` loads assets per app and how Matplotlib’s BytesIO converts charts to a base64 string for Django template embedding.”

Key Takeaway: The separation of concerns in Django, from ORM to views to templates, makes it powerful for integrating different libraries like Matplotlib and Plotly to build data-driven UIs.